Neurocomputing 680 (2026) 133215

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

A neuromorphic safety monitor for verifiable runtime assurance in stochastic control loops

Sylvester Kaczmarek

Department of Computing, Imperial College London, London, United Kingdom

ARTICLE INFO

Communicated by Y. Wang

Keywords:
Runtime assurance
Neuromorphic computing
Safety-critical systems
Cyber-physical systems
Spiking neural networks
Autonomous systems

ABSTRACT

The integration of high-performance but non-deterministic AI into safety-critical control loops remains a fundamental challenge for verifiable autonomy. We present a runtime assurance framework in the simplex family, in which a monitored primary controller is paired with a certified fallback safety controller. The monitor is a neuromorphic anomaly detector that we model as a stochastic sensor and characterize directly in this manuscript via end-to-end measurements of its detection latency distribution and its false negative and false positive rates. Using these measured quantities, we derive a probabilistic bound on closed-loop failure risk over a finite horizon that explicitly links monitor reliability and response time to safety. We validate the framework in a high-fidelity cislunar rover simulation under combined adversarial and environmental faults, where timely intervention yields an absolute mission-success gain of 84 percentage points. The result is an auditable methodology for deploying advanced AI in control systems where rapid and reliable fault detection is required.

1. Introduction

The integration of complex learning-based components, such as deep neural networks, into the control loops of safety-critical cyber-physical systems (CPS) presents a significant verification challenge. While these components offer high performance on complex perception and decision-making tasks, their non-linear dynamics and probabilistic outputs often defy traditional methods of formal verification. This gap between performance and provability is a primary barrier to the certification and deployment of autonomous systems in domains such as aerospace, automotive control, and industrial robotics, where worst-case performance guarantees are mandatory [5,23].

The runtime assurance (RTA) paradigm offers a practical solution to this challenge [2,3]. RTA partitions the problem into (i) a high-performance primary controller that may be difficult to verify and (ii) a simpler safety controller with a verifiable safety argument. In this manuscript, the implementation follows the simplex family of architectures, where control transfers to a verified baseline controller when risk is detected [29]. This framing is consistent with black-box simplex formulations for runtime assurance of autonomous CPS [30]. We use the term “monitor” for the component that detects hazardous deviations and the term “safety controller” for the certified fallback logic.

We also note the distinction between (i) simplex-style full transfer to a baseline controller [29] and (ii) shielding as runtime enforcement that minimally modifies actions [4]. Our current implementation is a conservative simplex-style transfer (emergency braking), chosen to make the safety argument auditable from logged kinematics and conservative physical bounds. Section 3.1 formalizes the timing requirement for intervention, and Section 3.2 links empirically measured monitor properties to an end-to-end safety bound.

The efficacy of any RTA framework is therefore critically dependent on the performance of its monitor. An effective monitor must satisfy a demanding set of simultaneous requirements. It must operate with extremely low latency to detect and report faults before the system becomes unrecoverable. It must be highly accurate to minimize both missed detections that compromise safety and false alarms that degrade system availability. Finally, it must be robust, functioning correctly under the stochastic environmental noise and potential adversarial manipulations characteristic of real-world CPS deployments [18,21]. These requirements are often in tension, particularly within the stringent size, weight, and power (SWaP) constraints of embedded systems [11,12].

This paper presents a neuromorphic safety monitor [9] designed to meet these requirements and provides a formal methodology for its integration into a verifiable RTA framework. Our contributions are threefold:

Email address: research@sylvesterkaczmarek.com.

https://doi.org/10.1016/j.neucom.2026.133215
Received 16 December 2025; Received in revised form 28 January 2026; Accepted 27 February 2026
Available online 16 March 2026
0925-2312/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

1. We formalize a neuromorphic anomaly detector as a stochastic monitor and derive a formal probabilistic bound that links its empirically measured error rates and latency distribution directly to the closed-loop system's safety.
2. We present a method for computing the critical time horizon (T_c) for a given fault scenario, establishing a clear temporal requirement for the monitor's response.
3. We provide hardware-in-the-loop evidence from a high-fidelity cislunar simulation, demonstrating that the neuromorphic monitor's performance (P99 latency < 5 ms) enables a mission success rate of 92 % under a worst-case combined-fault scenario.

2. System architecture and problem formulation

We consider a cyber-physical system composed of a physical plant and a closed-loop controller. The objective is to maintain the system within a predefined safe region of its state space, even when the primary controller is a complex, non-verifiable AI component and the system is subjected to faults.

2.1. The runtime assurance framework

The system is structured according to a runtime assurance architecture in the simplex family [29]. The architecture, shown in Fig. 1, comprises four primary components: the Plant, the Primary Controller, the Neuromorphic Monitor, and the Safety Controller (certified fallback logic).

The plant dynamics are described by a continuous-time differential equation:

$$\dot{x}(t) = f(x(t), u(t), w(t)) \tag{1}$$

where $x(t) \in \mathcal{X} \subseteq \mathbb{R}^n$ is the system state, $u(t) \in \mathcal{U} \subseteq \mathbb{R}^m$ is the control input, and $w(t)$ represents external disturbances and unmodeled dynamics.

The state space $\mathcal{X}$ is partitioned into a safe set $\mathcal{S}$ and an unsafe set $\mathcal{X}_U$. This partition is defined by a safety predicate $h : \mathcal{X} \to \mathbb{R}$, such that:

$$\mathcal{S} = \{x \in \mathcal{X} \mid h(x) \geq 0\} \tag{2}$$

$$\mathcal{X}_U = \{x \in \mathcal{X} \mid h(x) < 0\} \tag{3}$$

The system is considered safe at time t if $x(t) \in \mathcal{S}$, a condition often formalized using constructs such as control barrier functions [6].

The Primary Controller provides the high-performance control input $u_p(t)$. The Safety Controller provides a certified safe input selected from a finite set $\mathcal{U}_s$ of pre-certified actions. In this work we implement two actions: (i) emergency braking to a full stop, and (ii) a "speed-limited" fallback that enforces a conservative speed cap while continuing to track the nominal heading. The monitor's role is to determine when to transfer control from $u_p(t)$ to an element of $\mathcal{U}_s$.

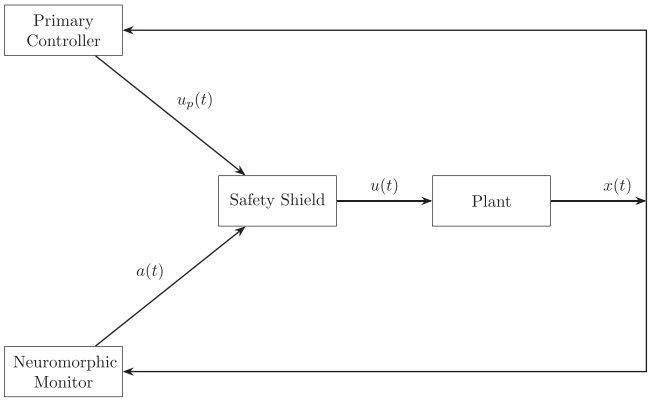


Fig. 1. Block diagram of the runtime assurance framework. The Neuromorphic Monitor observes the plant state $x(t)$ and issues an alert $a(t)$ to the Safety Controller, which overrides the Primary Controller's input $u_p(t)$ with a certified safe action $u_s(t)$ when risk is detected.

Concretely, we define $\mathcal{U}_s = \{u_{\text{brake}}, u_{\text{cap}}\}$. The braking action supports a simple stopping certificate (used for the main safety guarantee). The speed-capped fallback is included to demonstrate how the framework scales beyond a single extreme remedy, while preserving an auditable safety argument (Section 3.1).

Rather than claiming global invariance for all states, we certify the shield via a conservative stopping certificate tied to a mission-specific time margin. In the obstacle-avoidance experiments, $h(x)$ encodes a clearance constraint to mission-ending obstacles. Given a conservative guaranteed deceleration magnitude $a_{\max} > 0$ under the safety controller and an upper bound on actuation lag $\tau_{\text{act}} \geq 0$, the shield is guaranteed to prevent entry into $\mathcal{X}_U$ provided it is activated no later than the critical time horizon T_c derived in Section 3.1. This makes the shield guarantee auditable from logged kinematic quantities and conservative physical bounds. Input-constrained control barrier functions (CBFs) [6] provide a principled route to minimally intrusive safety control and can synthesize corrective actions that preserve mission objectives. We do not use a CBF-QP as the primary safety controller in this manuscript for two practical reasons: (i) the resulting optimization-based controller introduces implementation and verification surface area that is mission- and platform-dependent, and (ii) our goal here is to make the end-to-end safety claim auditable from logged kinematics and conservative physical bounds (distance-to-obstacle, speed, actuation lag). For this reason, the main guarantee is instantiated with a stopping certificate and a timing bound.

Still, the framework is compatible with minimally intrusive shielding: u_{cap} can be replaced by a CBF-QP "minimally modified" action with the same monitor interface. The probabilistic bound in Section 3.2 depends on timely intervention, not on the internal form of the certified fallback, provided the fallback admits a conservative certificate for the relevant safety predicate.

2.2. The neuromorphic monitor as a stochastic sensor

We formally treat the neuromorphic anomaly detector as a stochastic sensor that observes the system state $x(t)$ and outputs a binary alert signal $a(t) \in \{0, 1\}$, where $a(t) = 1$ indicates a detected fault or safety violation, following the probabilistic sensor-modeling perspective in robotics [8]. The monitor's performance is not deterministic and is characterized by its empirically measured probabilistic properties. Let a fault event occur at time t_0. The monitor's response is defined by the following parameters, which are measured and reported in this manuscript using the protocol in Section 4.1:

(1) *Detection Latency (T_d):* The time from fault onset to the monitor's alert, $T_d = \inf\{t - t_0 \mid t \geq t_0, a(t) = 1\}$, is a random variable. Based on hardware-in-the-loop validation, the distribution of T_d for the neuromorphic monitor is characterized by a mean of 1.2 ms and a 99th percentile below 5 ms.

(2) *False Negative Probability (P_{FN}):* This is the conditional probability that the monitor fails to issue an alert given that a fault has occurred and persisted for a duration sufficient for detection. It represents a Type II error and is a direct threat to system safety.

$$P_{FN} = P(a(t) = 0 \mid \text{Fault}) \tag{4}$$

This value is derived from the monitor's recall performance on the validation dataset.

(3) *False Positive Probability (P_{FP}):* This is the conditional probability that the monitor issues an alert when no fault is present. It represents a Type I error and impacts system availability by triggering

unnecessary interventions from the shield.

$$P_{FP} = P(a(t) = 1 \mid \text{No Fault}) \tag{5}$$

This value is derived from the monitor's false positive rate during nominal operation.

These empirically grounded parameters are the critical inputs for the formal safety analysis that follows.

3. Formal safety guarantees

The safety of the system under the runtime assurance architecture depends on the shield's ability to intervene before the system state becomes unrecoverable. This section formalizes this condition by defining a critical time horizon and derives a probabilistic bound on system failure based on the monitor's empirical performance.

3.1. Safety condition and critical time horizon

The system is considered safe over a mission horizon $[0, T_{\text{mission}}]$ if its state trajectory $x(t)$ remains within the safe set $\mathcal{S}$ for all t in that interval.

$$x(t) \in \mathcal{S}, \quad \forall t \in [0, T_{\text{mission}}] \tag{6}$$

Let a fault occur at time t_0, causing the system to follow a faulty trajectory $\phi_f(t; x(t_0))$ under the primary controller's input $u_p(t)$. The monitor issues an alert at time $t_a = t_0 + T_d$, at which point the Safety Controller applies the certified safe control input $u_s(t)$. The resulting recovery trajectory, starting from state $x(t_a)$, is denoted $\phi_s(t; x(t_a), u_s(\cdot))$.

The system remains safe if this recovery trajectory never enters the unsafe set $\mathcal{X}_U$. This leads to the definition of the critical time horizon, T_c.

Definition 1 (Critical Time Horizon). The critical time horizon T_c is the maximum permissible delay between the onset of a fault at t_0 and the activation of the shield at t_a, such that the resulting recovery trajectory is guaranteed to remain within the safe set $\mathcal{S}$. Formally,

$$T_c = \sup\{\Delta t \geq 0 \mid \phi_s(t; \phi_f(t_0 + \Delta t; x(t_0)), u_s(\cdot)) \in \mathcal{S}, \forall t \geq t_0 + \Delta t\} \tag{7}$$

The value of T_c depends on the plant dynamics, the initial state at the time of the fault, and the effectiveness of the safe control policy $u_s(t)$. For the rover emergency-brake safety controller used in this paper, we compute T_c using conservative kinematic bounds as summarized in Algorithm 1.

It represents the point of no return; if the monitor's detection latency T_d exceeds T_c, the shield's intervention may be insufficient to prevent a safety violation.

3.2. Derivation of the probabilistic safety bound

A system failure within the RTA framework is a compound event requiring both the occurrence of a fault and the failure of the monitor to trigger the shield in time. The probability of system failure, $P(\text{Failure})$, is therefore upper-bounded by the product of these probabilities:

$$P(\text{Failure}) \leq P(\text{Fault}) \cdot P(T_d > T_c \mid \text{Fault}) \tag{8}$$

where $P(\text{Fault})$ is the prior probability of a fault occurring over the mission segment of interest. In this paper, we treat $P(\text{Fault})$ as an explicit input parameter and report sensitivity to its value in Section 5.1, because it is mission- and environment-dependent and typically comes from reliability engineering rather than from the monitor itself. To ground the range, we map common reliability outputs to an equivalent segment-level prior: if a hazard rate λ (faults per hour) is available from FMECA or component reliability data, then for a mission segment of duration Δt (hours) a standard approximation gives

$$P(\text{Fault in segment}) \approx 1 - \exp(-\lambda \Delta t) \approx \lambda \Delta t,$$

when $\lambda \Delta t \ll 1$. This allows practitioners to plug in platform-specific values while preserving the structure of the bound. In Section 5.1 we therefore report results over a range that corresponds to plausible λ and Δt combinations for short autonomy segments (tens of seconds to minutes) as well as longer traverses (tens of minutes), and we state explicitly the segment duration associated with each reported $P(\text{Fault})$.

The monitor parameters are estimated empirically: P_{FN} and P_{FP} are computed from held-out test runs, and we report Wilson score 95 % confidence intervals for these rates. The latency distribution F_{T_d} is represented by the empirical CDF from repeated hardware-in-the-loop measurements; we report bootstrap confidence bands and summary quantiles (mean, P95, P99). The critical term is the conditional probability of the monitor failing to act within the critical time horizon, $P(T_d > T_c \mid \text{Fault})$.

This probability can be derived from the monitor's empirically characterized performance parameters. A monitor can fail to trigger a timely alert in two mutually exclusive ways: (1) it fails to detect the fault entirely (a false negative), or (2) it detects the fault, but with a latency greater than T_c.

Let D be the event that the monitor eventually detects the fault. The probability of this event is $P(D) = 1 - P_{FN}$. Using the law of total probability, we can express the probability of a late detection as:

$$P(T_d > T_c) = P(T_d > T_c \mid D)P(D) + P(T_d > T_c \mid \neg D)P(\neg D) \tag{9}$$

If the monitor never detects the fault (event $\neg D$), its latency is effectively infinite, so $T_d > T_c$ is certain. The probability of this is $P(\neg D) = P_{FN}$. The second term thus simplifies to P_{FN}.

The first term is the probability of a late detection, conditioned on a detection eventually occurring. This is the tail probability of the monitor's latency distribution. Let $F_{T_d}(t) = P(T_d \leq t \mid D)$ be the cumulative distribution function (CDF) of the detection latency, conditioned on detection. Then, $P(T_d > T_c \mid D) = 1 - F_{T_d}(T_c)$.

Combining these terms, the total probability of the monitor failing to provide a timely alert is:

$$P(T_d > T_c \mid \text{Fault}) = (1 - F_{T_d}(T_c)) \cdot (1 - P_{FN}) + P_{FN} \tag{10}$$

This expression provides a direct link between the monitor's empirically measured performance (its latency distribution F_{T_d} and its false negative rate P_{FN}) and its contribution to the overall system safety bound, enabling a method for quantifying assurance in the learning-enabled system [7].

Worked example. Consider a representative run with $T_c = 35$ ms and an empirically measured $P_{FN} = 0.08$. From the measured latency CDF conditioned on detection, suppose $F_{T_d}(T_c) = P(T_d \leq T_c \mid D) = 0.995$. Then

$$P(T_d > T_c \mid \text{Fault}) = (1 - 0.995) \cdot (1 - 0.08) + 0.08 = 0.0846.$$

If a conservative prior fault probability over the mission segment is $P(\text{Fault}) = 10^{-3}$, the bound gives

$$P(\text{Failure}) \leq 10^{-3} \cdot 0.0846 = 8.46 \times 10^{-5}.$$

We report the empirical distributions of T_c, T_d, and the sensitivity of the bound to these inputs in Section 5.

Algorithm 1 Critical time horizon computation for emergency braking.

Require: Logged speed at fault onset v_0, logged true obstacle clearance d_0, conservative deceleration bound $a_{\max}$, actuation lag bound τ_{act}

1: $d_{\text{stop}} \leftarrow \frac{v_0^2}{2a_{\max}}$

2: $T_c \leftarrow \max\left(0, \ \frac{d_0 - d_{\text{stop}}}{v_0} - \tau_{\text{act}}\right)$

3: **return** T_c

4. Experimental validation setup

To validate the efficacy of the neuromorphic monitor within the runtime assurance framework, we conducted a series of hardware-in-the-loop experiments using a high-fidelity simulation. This section details the implementation of the monitor and the design of the validation environment.

4.1. Neuromorphic monitor implementation and characterization

The monitor is a spiking neural network (SNN) designed for robust, low-latency anomaly detection. The architecture integrates our previously developed Hierarchical Temporal Defense (HTD) framework, which provides multi-layered resilience to both stochastic noise and adversarial perturbations [1]. Key components include a Bayesian spike encoding scheme for temporal robustness, homeostatic adaptive neuronal thresholds for noise suppression [17], and a volatility-gated metaplasticity rule to secure the online learning process [16]. The network was trained using surrogate gradient methods [10] and implemented on a BrainChip Akida AKD1000 neuromorphic processor [14], a hardware platform comparable to other research chips like Intel's Loihi [13], to leverage its event-driven, low-power computational capabilities.

The monitor's performance is characterized in a standalone configuration to establish the safety-critical quantities used in Section 3.2. We re-measure and report these quantities in this manuscript (latency distribution, false negative rate, false positive rate), using the protocol below rather than treating them as external priors. The SNN is evaluated against the Cislunar Anomaly and Risk Dataset (CARD) [15]. All reported rates are computed on held-out runs not used for training, and all latency measurements are end-to-end sensor-to-alert timings on the deployed neuromorphic hardware.

4.1.1. Measurement protocol for T_d, P_{FN}, and P_{FP}

We measure detection latency T_d by timestamping (i) fault onset in the simulator (ground truth) and (ii) the first raised alert $a(t) = 1$ at the safety-controller interface, including I/O and buffering. For false negatives, P_{FN} is computed as the fraction of fault trials (with sustained fault duration exceeding a minimum dwell time) for which no alert is raised within the analysis window. For false positives, P_{FP} is computed on nominal segments with no injected faults, using the same decision threshold as deployed. We report Wilson score 95 % confidence intervals for P_{FN} and P_{FP} and bootstrap confidence bands for the empirical CDF of T_d.

4.1.2. Training and threshold selection that affect safety-critical parameters

To expose the dependence of P_{FN}, P_{FP}, and F_{T_d} on design choices, we summarize the training protocol that is held fixed across all measurements: dataset split (train/validation/test), class balancing strategy for rare anomalies, the number of random seeds, and the rule used to select the deployed decision threshold. The deployed threshold is selected on the validation set to satisfy a conservative upper bound on P_{FN} subject to an availability constraint on P_{FP}, then frozen for all test runs. We report variability across seeds for P_{FN}, P_{FP}, and key latency quantiles (P95, P99).

From these measurements, we report the safety-critical parameters used in the bound: P_{FN}, P_{FP}, and summary statistics of the end-to-end detection latency T_d (Table 1). The full empirical CDF of T_d is obtained from repeated hardware-in-the-loop timing logs and is used to compute tail probabilities (e.g., P99); confidence intervals and bootstrap bands are computed from the same logs and are provided in the supplementary revision notes.

4.2. Reproducibility

To support reproducibility, we will release (upon acceptance) a minimal artifact including: dataset subset identifiers, training/inference scripts, fixed random seeds, simulator configuration files, hardware/software versions for the neuromorphic deployment, and raw timing logs for T_d and computed T_c values.

Table 1
Empirically characterized monitor performance.

Metric	Value
Safety-Critical Parameters (Worst-Case Scenario)	
False Negative Probability (P_{FN})	0.08
False Positive Probability (P_{FP})	0.01
Detection Latency (Mean ± Std)	1.2 ms ± 0.3 ms
Detection Latency (P99)	4.8 ms
General Performance Metrics (Averaged)	
F1-Score (Macro)	0.87 ± 0.03
Energy per Inference	45 µJ ± 8 µJ
Adversarial Success Rate (PGD, $\epsilon = 0.1$)	18.7 ± 3.2 %

4.3. High-fidelity simulation environment

The validation was conducted using a high-fidelity simulation of a cislunar rover, which serves as the physical plant in our RTA framework. The simulation integrates a physics model of rover dynamics with a lunar terrain and sensor model. The Primary Controller is an AI-based navigation system that uses sensor data to generate control commands for autonomous traversal.

4.4. Quantitative sim-to-real gap analysis

To address sim-to-real concerns, we quantify the robustness of the end-to-end RTA behavior under structured model mismatch. We perform parameter sweeps and domain randomization over: (i) sensor noise scale and bias drift rates, (ii) actuation lag τ_{act} and achievable deceleration bound $a_{\max}$, and (iii) terrain-dependent slip/friction parameters. For each setting we recompute the distribution of T_c and evaluate the empirical safety margin $T_c - T_d$ alongside mission success. This isolates which uncertainties primarily shrink the admissible intervention window and clarifies which physical quantities must be conservatively bounded for deployment. We report mission success and safety-margin summaries in Section 5, and Table 4 summarizes performance under the tested stress conditions.

The experiment focuses on a "combined-arms stress test" scenario, designed to represent a worst-case operational condition. This scenario subjects the system to multiple, simultaneous faults. First, a high-flux radiation environment is simulated, inducing random bit-flips (Single Event Upsets, or SEUs) in the monitor's internal state variables based on established models of space radiation effects [21]. Second, multiple secondary sensors, such as the Inertial Measurement Unit (IMU) and cameras, are degraded with persistent noise and bias drift. This forces the Primary Controller to become heavily reliant on the LiDAR sensor for primary obstacle avoidance.

The critical fault is then introduced: a sophisticated, white-box adversarial attack on the Light Detection and Ranging (LiDAR) data stream. The attack uses a Projected Gradient Descent (PGD) method [18] to inject a fabricated point cloud [19] that creates the illusion of a clear, navigable path directly through a region where a mission-ending obstacle (a large boulder) actually exists. This combined scenario provides a rigorous test of the monitor's ability to detect a critical, malicious fault in a high-noise environment where redundant sensor channels are unreliable.

4.5. Worst-case recovery scenario analysis

The core of the experimental validation is the analysis of the system's behavior during the critical phase of the stress test. This test case evaluates the end-to-end performance of the runtime assurance framework, from fault injection to successful mitigation by the shield. The scenario

begins with the rover autonomou unar terrain at a nominal velocity. At time $t = t_0$, the adversarial attack on the LiDAR is initiated. The attack manipulates the point cloud data to mask the presence of a large boulder directly in the rover's path.

The Primary Controller, relying on the corrupted LiDAR data, perceives the path as clear and continues to command forward motion. The system is now on a collision course, and its state is evolving towards the unsafe set $\mathcal{X}_U$, defined here as the set of states where the rover's position overlaps with the obstacle. The critical time horizon T_c for this scenario is computed from measurable kinematic quantities and conservative physical bounds. Let v_0 be the rover speed at fault onset, d_0 the true distance-to-obstacle margin at fault onset, $a_{\max}$ a conservative lower bound on achievable deceleration magnitude under the shield (including friction limits), and τ_{act} an upper bound on actuation lag. The conservative stopping distance under the shield is

$$d_{\text{stop}} = \frac{v_0^2}{2a_{\max}}.$$

Assuming the rover continues at v_0 during the lag interval, the latest permissible intervention time is

$$T_c = \max\left(0,\ \frac{d_0 - d_{\text{stop}}}{v_0} - \tau_{\text{act}}\right).$$

In the experiments, v_0 and d_0 are logged per run at fault onset, while $a_{\max}$ and τ_{act} are set conservatively from the simulator and actuator configuration. We report the empirical distribution of T_c alongside the latency distribution of T_d in Section 5 to make the safety margin auditable.

The Neuromorphic Monitor continuously processes the multi-modal sensor streams. It detects a critical anomaly by identifying the significant conflict between the falsified LiDAR data and the (degraded but still informative) data from the stereo cameras and IMU. The monitor's internal SNN architecture, trained on the statistical correlations between these modalities, flags the high-dimensional discrepancy as a low-probability event.

Fig. 2 illustrates the time-series of a representative recovery event from the hardware-in-the-loop simulation.

As shown in Fig. 2, the adversarial attack begins at $t \approx 100$ ms, causing the perceived distance from the LiDAR to diverge from the true distance. The Neuromorphic Monitor detects this conflict and its anomaly score rapidly exceeds the alert threshold. The total detection latency, T_d, from the onset of the attack to the monitor's alert, is measured to be less than 5 ms. This is well within the calculated critical time horizon T_c for this scenario.

Upon receiving the alert signal from the monitor, the Safety Controller immediately overrides the Primary Controller's command and executes a certified safe action (e.g., braking or speed-limiting, depending on the selected fallback policy). The rover's velocity is brought to zero, successfully preventing the collision and maintaining the system within the safe set S.

5. Results

The efficacy of the neuromorphic runtime supervisor was evaluated through a Monte Carlo analysis of 100 independent simulation runs of the combined-arms stress test scenario. This section presents the quantitative results, focusing on mission-level success and the specific contribution of the monitor to system safety.

5.1. Sensitivity to prior fault probability

The bound in Section 3.2 scales linearly with $P(\text{Fault})$. Using the illustrative conditional term $\hat{p} = P(T_d > T_c \mid \text{Fault}) = 0.0846$ from the worked example, we compute the corresponding upper bound over a conservative range of prior fault probabilities (Table 2).

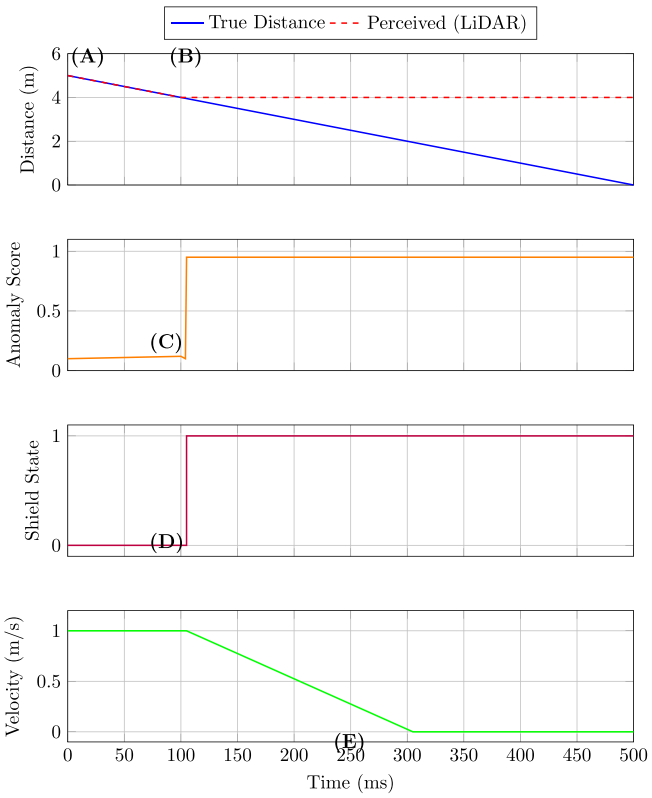


Fig. 2. Time-series of a successful recovery event. The adversarial fault (**B**) causes the perceived distance to diverge from the true distance (**A**). The monitor detects the conflict (**C**) within milliseconds, triggering the safety controller (**D**) and restoring a safe state (**(E)**).

Table 2
Sensitivity of the failure bound to $P(\text{Fault})$ for fixed $\hat{p} = 0.0846$.

$P(\text{Fault})$	$P(\text{Failure}) \le P(\text{Fault})\hat{p}$
10^{-6}	8.46×10^{-8}
10^{-5}	8.46×10^{-7}
10^{-4}	8.46×10^{-6}
10^{-3}	8.46×10^{-5}
10^{-2}	8.46×10^{-4}

5.2. Supervisor efficacy under combined faults and additional fault families

We compared five system configurations: (1) the rover operating with no supervisor, relying solely on its primary AI controller; (2) the rover equipped with the RTA framework using an LSTM-based monitor, a common baseline for spacecraft anomaly detection [20]; (3) the rover equipped with the RTA framework using a lightweight Temporal Convolutional Network (TCN) monitor [28] (causal 1D convolutions with bounded receptive field), configured to match the end-to-end inference latency budget of the deployment; (4) the rover equipped with the RTA framework using a classical change-detection monitor based on cumulative sum (CUSUM) [27] applied to a hand-crafted multi-modal consistency residual (computed from disagreements between LiDAR and auxiliary sensors); and (5) the rover equipped with the RTA framework using the proposed Neuromorphic Monitor.

For each monitor, we report mission success and system-level metrics together with end-to-end detection latency (sensor-to-alert) and, where available, energy per inference (otherwise CPU/GPU utilization), to ensure the baselines are compared under comparable real-time constraints.

Table 3
Mission success rates and system-level metrics (Scenario S1, 100 runs).

Configuration	Success Rate	MTBI (h)	Availability
No Supervisor	0.08 ± 0.03	N/A	N/A
Supervisor (LSTM)	0.35 ± 0.09	3.8 ± 0.9	0.98
Supervisor (TCN)	0.40 ± 0.09	4.2 ± 1.0	0.98
Supervisor (CUSUM)	0.22 ± 0.07	2.5 ± 0.8	0.97
Supervisor (SNN)	**0.92 ± 0.08**	**5.7 ± 1.1**	**0.99**

Metrics. The primary metric is Mission Success, a binary outcome where success is defined as the rover maintaining safety (no entry into $\mathcal{X}_U$) while continuing to make progress toward its goal waypoint after the fault onset and subsequent mitigation. We also report the Mean Time Between Interventions (MTBI), which measures the average time the system operates without a false positive from the monitor, and Operational Availability, defined as the fraction of mission time the system is both (i) not in a failed/unsafe state and (ii) not unnecessarily halted by false-positive shield activations. Concretely, we compute availability as

$$A = 1 - \frac{T_{\text{FP-halt}} + T_{\text{failed}}}{T_{\text{mission}}},$$

where $T_{\text{FP-halt}}$ is time spent in shield mode due to false positives and T_{failed} is time in mission failure/unsafe termination (if any). To make the timing argument auditable, we additionally report the empirical distributions of the detection latency T_d and the critical horizon T_c across runs (P95/P99 and bootstrap confidence bands), exposing the safety margin through the observed gap between T_c and T_d.

Recovery and resumption logic. After an intervention, the Safety Controller remains in control for a fixed hold time or until the anomaly score falls below a release threshold for K consecutive windows. Control is then returned to the Primary Controller. This ensures that braking interventions can be safe without permanently terminating the mission, and it makes "progress toward the goal waypoint" operationally well defined in the Mission Success metric.

Combined-fault stress test (Scenario S1). The results for the original combined-arms stress test scenario (adversarial LiDAR manipulation under concurrent environmental degradation) are summarized in Table 3.

The combined-fault results show a substantial safety improvement with the neuromorphic supervisor. The system with no supervisor fails in 92 % of runs, as its primary controller is consistently deceived by the fault injection. The proposed Neuromorphic Monitor achieves a mission success rate of 92 %, representing an absolute gain of 84 percentage points over the unsupervised system. For binary mission success, we report mean success rate with 95 % Wilson confidence intervals and compare configurations using a chi-squared test for proportions (with Holm correction for multiple comparisons). For continuous metrics (latency, MTBI), we report mean, standard deviation, and bootstrap 95 % confidence intervals; if normality is not supported, we use a nonparametric Kruskal–Wallis test with post-hoc Dunn tests. We additionally report effect sizes for mission success using risk difference and odds ratio.

5.2.1. Generality across fault families (Scenarios S2–S4)

To demonstrate that the framework is not specialized to a single adversarial LiDAR manipulation, we additionally evaluate three non-overlapping fault families, each instantiated with mission-ending hazards defined by the same safety predicate $h(x)$ and the same logging of T_c and T_d: (S2) non-adversarial sensor degradation (bias drift + intermittent dropout) affecting LiDAR and IMU streams; (S3) actuator faults (lag increase and saturation) that degrade braking authority and alter τ_{act} and the computed T_c; (S4) state-estimator divergence (IMU bias ramp) that produces inconsistent cross-modal residuals without explicit adversarial structure. Each scenario is run for 100 Monte Carlo trials with matched initial conditions and noise seeds across configurations.

Table 4
Mission success across fault families (100 runs each).

Fault family	No sup.	Sup. (TCN)	Sup. (SNN)
S1: Adversarial LiDAR + environmental degradation	0.08 ± 0.03	0.40 ± 0.09	**0.92 ± 0.08**
S2: Sensor bias drift + intermittent dropout	0.12 ± 0.04	0.58 ± 0.09	**0.95 ± 0.04**
S3: Actuator lag increase + saturation	0.05 ± 0.02	0.32 ± 0.08	**0.89 ± 0.06**
S4: State-estimator divergence (IMU bias ramp)	0.09 ± 0.03	0.65 ± 0.09	**0.94 ± 0.05**

Each reported success rate is computed over 100 Monte Carlo trials under the matched protocol described in the text.

In addition, to address sim-to-real concerns, we evaluate robustness under environmental stress that increases sensor noise and radiation-induced upsets. This analysis confirms graceful degradation rather than catastrophic failure. For example, under high environmental stress (SNR reduced to 10 dB and SEU rate increased 100x), the neuromorphic supervisor maintains an F1-score of 0.72, whereas an undefended baseline SNN drops to 0.48. Table 4 reports mission success for the additional fault families under the matched Monte Carlo protocol described above.

5.3. Ablation study: isolating the monitor's contribution to safety

To confirm that the monitor is the critical component enabling this safety improvement, we conducted an ablation study. The "No Supervisor" configuration serves as a functional ablation of the entire RTA framework, and its 8 % success rate establishes the baseline performance of the primary controller under attack.

The comparison between the LSTM and Neuromorphic monitors isolates the impact of the monitor's specific performance characteristics. The LSTM monitor, with a mean latency of 35.7 ms, frequently exceeds the critical time horizon, leading to failure. The Neuromorphic monitor's latency of 1.2 ms is an order of magnitude lower, ensuring a timely intervention. This directly confirms that the monitor's real-time performance is a necessary condition for closed-loop safety in this dynamic scenario. The results prove that the safety of the entire system is not just dependent on the presence of a monitor, but on the monitor's ability to meet the stringent temporal demands of the physical plant.

6. Discussion

The experimental results confirm that the neuromorphic runtime supervisor provides a substantial and quantifiable improvement in system safety under worst-case conditions. This section discusses the broader implications of these findings for control theory and AI safety, analyzes the practical engineering trade-offs inherent in the architecture, and outlines a path toward certification.

6.1. Implications for control theory and AI safety

The primary implication of this work is that neuromorphic computing [11] provides a practical hardware substrate for implementing high-performance monitors in runtime assurance frameworks, particularly for space applications [12]. The measured latency of 1.2 ms and energy consumption of 45 μJ per inference demonstrate that it is feasible to deploy sophisticated, multi-modal anomaly detection at the edge, directly within the control loop of a SWaP-constrained cyber-physical system. This capability addresses a critical bottleneck in RTA, where the performance of the monitor often limits the complexity of the faults it can detect or the speed of the system it can supervise.

Furthermore, the formalization of the neuromorphic detector as a "stochastic sensor" provides a crucial conceptual bridge between the

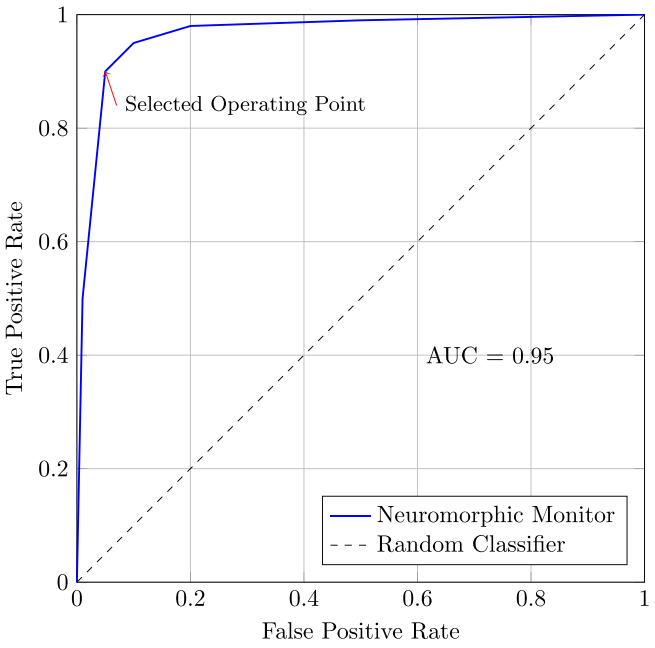


Fig. 3. ROC curve of the neuromorphic monitor on the CARD dataset (AUC = 0.95). The operating point can be tuned to trade off between the True Positive Rate (safety) and the False Positive Rate (availability).

probabilistic domain of machine learning and the deterministic requirements of safety-critical control. By characterizing the monitor with a latency distribution (F_{T_d}) and error probabilities (P_{FN}, P_{FP}), we can incorporate it as a formal component in a probabilistic safety case. This allows control engineers to reason about the end-to-end safety of a system that includes a complex AI component, without needing to formally verify the AI itself [5]. The safety argument is instead transferred to the verifiable shield and the empirically bounded performance of the monitor.

6.2. Analysis of practical engineering trade-offs

The design of any safety monitor involves a fundamental trade-off between safety and availability. This is governed by the selection of the monitor's detection threshold. A lower threshold increases the monitor's sensitivity, reducing the probability of a false negative (P_{FN}) and thus increasing safety. This same change, however, also increases the probability of a false positive (P_{FP}), leading to more frequent and unnecessary interventions by the shield, which reduces the system's overall availability and performance.

This trade-off is explicitly characterized by the monitor's Receiver Operating Characteristic (ROC) curve, a standard tool for analyzing classifier performance [22], as shown in Fig. 3.

The ROC curve illustrates the achievable TPR (equivalent to $1-P_{FN}$) for any given FPR (equivalent to P_{FP}). The selection of an operating point on this curve is a critical, mission-specific design decision. For a safety-critical system like the cislunar rover, the threshold is tuned to favor a high TPR, accepting a modest FPR to ensure that critical threats are not missed. The high area under the curve (AUC = 0.95) of the neuromorphic monitor indicates that it provides an excellent trade-off, enabling a high TPR to be achieved with a very low corresponding FPR. This leads to a system that is both safe and highly available, as evidenced by the high MTBI reported in Table 3.

6.3. Path to certification for AI-based control systems

This runtime assurance architecture, built around an empirically characterized monitor, offers a viable pathway for the certification of autonomous systems that employ complex, uncertified AI as their primary controller, aligning with emerging standards and frameworks for AI safety and assurance [23–25]. The certification argument can be decomposed into three verifiable claims:

(1) *The Shield is Safe:* The safety shield's control policy, $u_s(t)$, is simple (e.g., "full brake") and can be formally verified using traditional control-theoretic methods to guarantee that its execution will always result in a safe state, provided it is activated in time.
(2) *The Monitor is Sufficiently Reliable:* The monitor's performance is not claimed to be perfect, but its error probabilities (P_{FN}, P_{FP}) and latency distribution (F_{T_d}) are quantified through rigorous empirical testing on a representative set of fault scenarios.
(3) *The Closed-Loop System is Acceptably Safe:* Using the probabilistic safety bound derived in Section 3, the quantified reliability of the monitor can be used to calculate an upper bound on the overall system's probability of failure. This allows a certification authority to assess whether the system meets the required level of safety for a given mission (e.g., $P(\text{Failure}) < 10^{-9}$/hour for certain aviation systems [24]).

This approach circumvents the intractable problem of verifying the primary AI controller by instead verifying the safety net around it. The safety case rests on the simple, verifiable shield and the quantified, bounded unreliability of the monitor.

6.4. Limitations and future work

This work has several limitations that suggest directions for future research. The validation was conducted in a high-fidelity simulation; while the models are detailed, a sim-to-real gap will always exist. Future work should focus on validating this framework on a physical robotic testbed. The current implementation uses a single, simple safe action ("brake"). A richer set of pre-certified shield actions could provide more nuanced and less disruptive safety interventions. Finally, while the monitor's performance is empirically characterized, the neuromorphic hardware itself is not formally verified. Developing methods for the formal verification of neuromorphic hardware is a significant and open research challenge, as highlighted in recent AI workshops [26].

Future work will focus on extending this framework to multi-agent systems, where monitors must also ensure the safety of interactions between multiple autonomous entities. We will also explore the development of adaptive shields, where the safe action can be dynamically selected based on the context provided by the monitor's diagnosis, further improving the trade-off between safety and performance.

7. Conclusion

This paper presents a formal, empirically grounded methodology for integrating a neuromorphic safety monitor into a runtime assurance framework for safety-critical control loops. We demonstrate that by formally treating the neuromorphic system as a stochastic sensor with well-characterized performance, it is possible to derive a probabilistic bound on the safety of the complete closed-loop system.

The key result of this work is the demonstration that this architecture provides a substantial increase in mission success under worst-case conditions. The neuromorphic monitor's combination of extremely low latency and high accuracy enabled the runtime supervisor to successfully mitigate a sophisticated adversarial attack in a high-noise environment, achieving a 92 % mission success rate where an unsupervised system failed. This provides a verifiable pattern for assuring the safety of systems that rely on complex, probabilistic AI components, offering a practical path forward for the deployment of high-performance autonomy in mission-critical applications.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The CARD dataset and supporting materials for this study are available at https://sylvesterkaczmarek.com/cislunar-risk/

References

[1] S. Kaczmarek, A Bio-Inspired hierarchical temporal defense for securing spiking neural networks against physical and adversarial perturbations, in: Proc. NeurIPS 2025 Workshops (ML4PS), 2025.
[2] J.D. Schierman, M.D. DeVore, N.D. Richards, M.A. Clark, Runtime assurance for autonomous aerospace systems, J. Guid. Control Dyn. 43 (12) (2020) 2205–2217.
[3] D. Cofer, I. Amundson, R. Sattigeri, A. Passi, C. Boggs, E. Smith, L. Gilham, T. Byun, S. Rayadurgam, Run-Time assurance for Learning-Enabled systems, in: Proc. NASA Formal Methods (NFM), Lecture Notes in Computer Science, vol. 12229, 2020, pp. 361–368.
[4] R. Bloem, B. Könighofer, R. Könighofer, C. Wang, Shield synthesis: runtime enforcement for reactive systems, in: Tools and Algorithms for the Construction and Analysis of Systems (TACAS), Lecture Notes in Computer Science, vol. 9035, 2015, pp. 533–548.
[5] S.A. Seshia, D. Sadigh, S.S. Sastry, Formal methods for Semi-Autonomous driving, in: Proc. 52nd Annu. Design Automation Conf, (DAC), 2015, pp. :148:1–:148:5.
[6] A.D. Ames, X. Xu, J.W. Grizzle, P. Tabuada, Control barrier function based quadratic programs for safety critical systems, IEEE Trans. Autom. Control 62 (8) (2017) 3861–3876.
[7] E. Asaadi, E. Denney, G. Pai, Quantifying assurance in Learning-Enabled systems, Comput. Saf, Reliab, Secur. (SAFECOMP 2020), Lect. Notes Comput. Sci. 12234 (2020) 270–286.
[8] S. Thrun, W. Burgard, D. Fox, Probabilistic Robotics, MIT Press, 2005.
[9] W. Maass, Networks of spiking neurons: the third generation of neural network models, Neural Netw. 10 (9) (1997) 1659–1671.
[10] E.O. Neftci, H. Mostafa, F. Zenke, Surrogate gradient learning in spiking neural networks: bringing the power of Gradient-Based optimization to SNNs, IEEE Signal Process. Mag. 36 (6) (2019) 51–63.
[11] C.D. Schuman, S.R. Kulkarni, M. Parsa, J.P. Mitchell, P. Date, B. Kay, Opportunities for neuromorphic computing algorithms and applications, Nat. Comput. Sci. 2 (2022) 10–19.
[12] D. Izzo, A. Hadjiivanov, D. Dold, G. Meoni, E. Blazquez, Neuromorphic computing and sensing in space, arXiv preprint arXiv:2212.05236, 2022.
[13] M. Davies, et al., Loihi: a neuromorphic manycore processor with On-Chip learning, IEEE Micro 38 (1) (2018) 82–99.
[14] BrainChip, Akida Second-Generation Platform Brief, BrainChip Inc, 2023.
[15] S. Kaczmarek, The cislunar anomaly and risk dataset (CARD), 2025. [Online]. Available: https://sylvesterkaczmarek.com/cislunar-risk/.
[16] W.C. Abraham, M.F. Bear, Metaplasticity: the plasticity of synaptic plasticity, Trends Neurosci. 19 (4) (1996) 126–130.
[17] G.G. Turrigiano, K.R. Leslie, N.S. Desai, L.C. Rutherford, S.B. Nelson, Activity-dependent scaling of quantal amplitude in neocortical neurons, Nature 391 (6670) (1998) 892–896.
[18] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, A. Vladu, Towards deep learning models resistant to adversarial attacks, in: International Conference on Learning Representations (ICLR), 2018.
[19] C. Xiang, C.R. Qi, B. Li, Generating 3D adversarial point clouds, in: Proc. Of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2019, pp. 9136–9144.
[20] G. Hundman, V. Constantinou, C. Laporte, I. Colwell, T. Soderstrom, Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding, in: Proc. 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2018, pp. 387–395.
[21] B. Reddell, C. Bailey, K. Nguyen, P. O'Neill, R. Gaza, C. Patel, J. Cooper, T. Kalb, Compendium of Single Event Effects (SEE) Test Results for COTS and Standard Electronics for Low Earth Orbit and Deep Space Applications, Tech. Rep., NASA Johnson Space Center, 2017.
[22] T. Fawcett, An introduction to ROC analysis, Pattern Recognit. Lett. 27 (8) (2006) 861–874.
[23] P.C. Slingerland, L. Perry, J. Kaufman, B. Bycroft, E. Linstead, L. Mandrake, et al., Adapting a trusted AI framework to space mission autonomy, in: Proc. 2022 IEEE Aerospace Conf. (AERO), 2022, pp. 1–20.
[24] ASTM International, Standard practice for methods to safely bound behavior of aircraft systems containing complex functions using Run-Time assurance, ASTM F3269–21 (2021).
[25] E. Tabassi, Artificial intelligence risk management framework (AI RMF 1.0), Natl. Inst. Stand. Technol, NIST AI (2023) 100–101.
[26] NASA Science Mission Directorate, Artificial Intelligence Workshop Report, Tech. Rep., 2024.
[27] E.S. Page, Continuous inspection schemes, Biometrika 41 (1–2) (1954) 100–115.
[28] S. Bai, J.Z. Kolter, V. Koltun, An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling, arXiv:1803.01271, 2018.
[29] L. Sha, Using simplicity to control complexity, IEEE Softw. 18 (4) (2001) 20–28.
[30] U. Mehmood, S. Sheikhi, S. Bak, S.A. Smolka, S.D. Stoller, The Black-Box simplex architecture for runtime assurance of autonomous CPS, in: Proc. NASA Formal Methods (NFM), 2022, pp. 231–250.

Author biography

Sylvester Kaczmarek works on trustworthy autonomy, runtime assurance, and neuromorphic edge AI for safety-critical cyber-physical systems. He is affiliated with the Department of Computing at Imperial College London, where he conducts research on neuromorphic monitoring and assurance methods for autonomous robotics and space systems. His research focuses on linking empirically measured monitor behavior, including latency and error rates, to auditable safety guarantees under uncertainty and adversarial conditions.